ISPARTA UYGULAMALI BİLİMLER ÜNİVERSİTESİ
TEKNOLOJİ FAKÜLTESİ

ELEKTRİK ELEKTRONİK MÜHENDİSLİĞİ

MAKİNE ÖĞRENMESİNE GİRİŞ

Random Search ile Hiperparametre Optimizasyonu

DERSİN ÖĞRETİM ÜYESİ
Dr. Öğr. ALİ ŞENTÜRK

AD SOYAD : AYŞE NUR KÜÇÜKGÖZEL
NUMARASI : 2312705026
ÖĞRETİM : 1. ÖĞRETİM

Makine Öğrenmesinde Hiperparametre Optimizasyonu ve Batch Normalization Analizi

1. Giriş

Makine öğrenmesi, verilerden anlamlı sonuçlar çıkararak karar verme süreçlerini iyileştirmeyi amaçlayan bir alandır. Bu süreçte kullanılan modellerin başarısı yalnızca seçilen algoritmaya değil, aynı zamanda modelin eğitiminde kullanılan hiperparametrelere de bağlıdır. Hiperparametreler, model eğitilmeden önce belirlenen ve öğrenme sürecini doğrudan etkileyen ayarlardır. Bu parametrelerin uygun şekilde seçilmemesi durumunda, model istenilen performansı gösteremeyebilir.

Günümüzde veri miktarının artmasıyla birlikte daha doğru ve hızlı sonuçlar elde etmek önemli hale gelmiştir. Bu noktada hiperparametre optimizasyon yöntemleri devreye girmektedir. Özellikle Random Search gibi yöntemler, çok sayıda parametre kombinasyonunu verimli bir şekilde deneyerek en iyi sonucu bulmayı amaçlar. Bunun yanında, modelin eğitim sürecini iyileştirmek için kullanılan Batch Normalization tekniği de öğrenmenin daha kararlı ve hızlı gerçekleşmesini sağlamaktadır.

Bu çalışmada, hiperparametre seçiminin model performansı üzerindeki etkisi incelenmiş ve Random Search yöntemi ile parametre optimizasyonu gerçekleştirilmiştir. Ayrıca Batch Normalization kullanımının model doğruluğu ve eğitim süresi üzerindeki etkileri karşılaştırmalı olarak değerlendirilmiştir. Böylece, farklı yöntemlerin model başarısına katkısı daha açık bir şekilde ortaya konulmuştur.

2. Temel Kavramlar

Makine öğrenmesi modellerinin doğru ve verimli çalışabilmesi için bazı temel kavramların anlaşılması oldukça önemlidir. Bu çalışmada özellikle hiperparametre, hiperparametre optimizasyonu, Random Search ve Batch Normalization kavramları üzerinde durulmuştur.

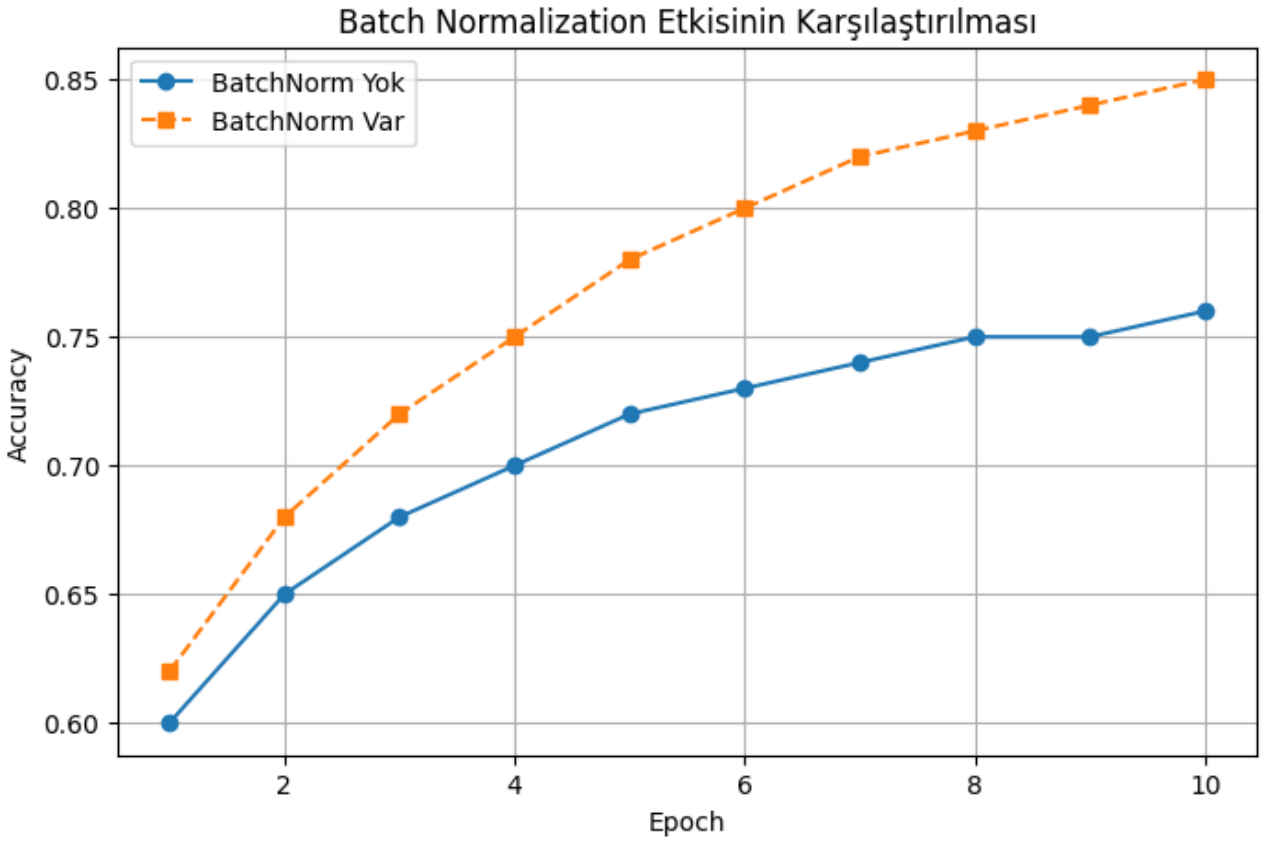
Hiperparametreler, model eğitimi başlamadan önce belirlenen ve öğrenme sürecini doğrudan etkileyen parametrelerdir. Learning rate (öğrenme oranı), batch size (veri grubu büyüklüğü) ve epoch sayısı bu parametrelere örnek olarak verilebilir. Bu değerlerin doğru seçilmesi, modelin daha hızlı öğrenmesini ve daha yüksek doğruluk elde etmesini sağlar. Yanlış belirlenen hiperparametreler ise modelin yetersiz öğrenmesine ya da aşırı öğrenmesine (overfitting) neden olabilir.

Hiperparametre optimizasyonu, model için en uygun parametre değerlerini bulmayı amaçlayan bir süreçtir. Bu süreçte farklı yöntemler kullanılmaktadır. Random Search yöntemi, belirlenen parametre aralığında rastgele seçimler yaparak farklı kombinasyonları dener ve en iyi sonucu veren parametre setini bulmaya çalışır.

Grid Search yöntemine göre daha az maliyetli olması ve geniş aralıkları daha hızlı tarayabilmesi açısından avantajlıdır.

Batch Normalization ise derin öğrenme modellerinde eğitim sürecini iyileştirmek amacıyla kullanılan bir tekniktir. Bu yöntem, her katmana gelen verileri normalize ederek dağılımın daha dengeli olmasını sağlar. Bu sayede model daha stabil bir şekilde öğrenir ve eğitim süresi kısalır. Ayrıca overfitting probleminin azalmasına da katkı sağlar.

Sonuç olarak, bu kavramlar makine öğrenmesi modellerinin performansını doğrudan etkileyen önemli bileşenlerdir. Doğru şekilde kullanıldıklarında modelin hem doğruluğunu hem de verimliliğini artırmaktadır.



Şekilde Batch Normalization kullanılan ve kullanılmayan modelin epochlara göre doğruluk değerleri karşılaştırılmıştır. Batch Normalization kullanılan modelin daha hızlı öğrenme sağladığı ve daha yüksek doğruluk değerlerine ulaştığı görülmektedir.

Matematiksel İfadeler ve Açıklamaları :

1. Batch Normalization :

$$\hat{x} = (x - \mu) / \sqrt{(\sigma^2 + \varepsilon)}$$

$$y = \gamma\hat{x} + \beta$$

Açıklama:

- $x \rightarrow$ giriş verisi
- $\mu \rightarrow$ ortalama
- $\sigma^2 \rightarrow$ varyans
- $\varepsilon \rightarrow$ çok küçük sayı
- $\gamma \rightarrow$ ölçekleme
- $\beta \rightarrow$ kaydırma

2. Softmax :

P(yi) = e^(zi) / (e^(z1) + e^(z2) + ... + e^(zn))

Açıklama:

- zi → model çıktısı
- P(yi) → olasılık

3. Loss (Cross Entropy):

$$\mathbf{L = -\Sigma\ (yi \log(\hat{y}i))}$$

Açıklama:

- yi → gerçek değer
- ŷi → tahmin

3. Yöntem / Model Açıklaması

Bu çalışmada, makine öğrenmesi modelinin performansını artırmak amacıyla iki farklı yaklaşım ele alınmıştır. İlk olarak modelin eğitim sürecinde etkili olan hiperparametrelerin en uygun değerlerini belirlemek için Random Search yöntemi kullanılmıştır. İkinci olarak ise Batch Normalization tekniğinin modelin öğrenme sürecine olan etkisi incelenmiştir. Bu bölümde kullanılan yöntemin çalışma mantığı, uygulama adımları ve modelin genel yapısı açıklanmıştır.

Makine öğrenmesi modellerinde başarı oranı, yalnızca kullanılan algoritmaya değil, aynı zamanda eğitim sırasında seçilen parametrelere de bağlıdır. Learning rate, batch size ve epoch sayısı gibi hiperparametreler modelin öğrenme hızını ve doğruluğunu doğrudan etkilemektedir. Bu nedenle çalışmada, bu parametrelerin farklı değerleri denenerek en uygun kombinasyonun bulunması amaçlanmıştır. Bunun için Random Search yöntemi tercih edilmiştir.

Bu yöntem, belirlenen parametre aralıkları içerisinden rastgele seçimler yaparak farklı kombinasyonları test eder ve en iyi sonucu veren parametreleri belirler.

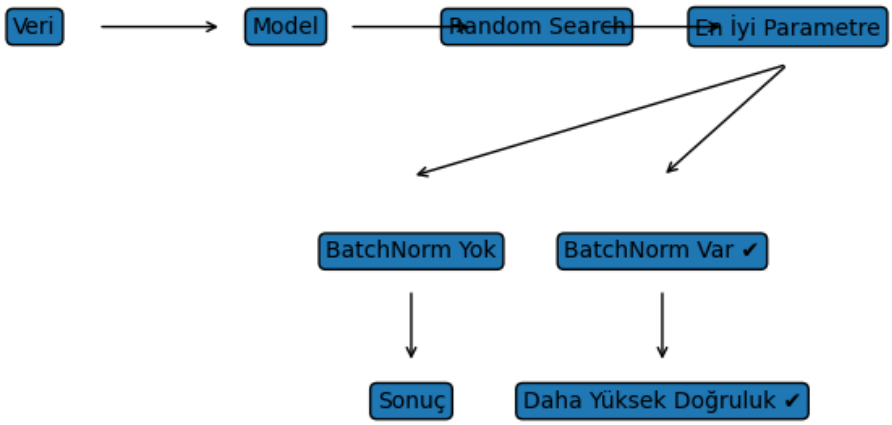
Modelin çalışma mantığı genel olarak şu şekildedir: Öncelikle veri seti eğitim ve test aşamalarında kullanılmak üzere hazırlanır. Daha sonra model yapısı oluşturulur ve başlangıç hiperparametreleri belirlenir. Random Search yöntemi ile farklı parametre kombinasyonları denenir. Her kombinasyon için model eğitilir ve elde edilen doğruluk sonuçları karşılaştırılır. En iyi sonucu veren parametreler seçildikten sonra aynı model bir kez Batch Normalization kullanılmadan, bir kez de Batch Normalization eklenerek eğitilir. Son aşamada her iki modelin doğruluk, hata oranı ve öğrenme kararlılığı bakımından karşılaştırması yapılır.

Adım adım uygulama süreci incelendiğinde ilk aşamada veri seti sisteme tanımlanmış ve modele uygun hale getirilmiştir. İkinci aşamada temel model yapısı kurulmuştur. Üçüncü aşamada Random Search yöntemi ile learning rate, batch size ve epoch gibi hiperparametreler için farklı değerler denenmiştir. Dördüncü aşamada en uygun parametre kombinasyonu seçilmiştir. Beşinci aşamada aynı model Batch Normalization katmanı eklenmeden eğitilmiştir. Altıncı aşamada ise modele Batch Normalization eklenerek tekrar eğitim gerçekleştirilmiştir. Son olarak elde edilen sonuçlar grafikler ve tablolar yardımıyla karşılaştırılmıştır.

Batch Normalization tekniğinin çalışma mantığı, her katmana gelen veriyi belirli bir dağılıma göre

normalize etmeye dayanmaktadır. Bu işlem sayesinde verinin dağılımındaki değişim azaltılır ve model daha dengeli bir öğrenme süreci geçirir. Böylece eğitim daha hızlı gerçekleşir, öğrenme süreci daha kararlı hale gelir ve bazı durumlarda daha yüksek doğruluk elde edilir. Bu nedenle çalışmada Batch Normalization kullanılan ve kullanılmayan model sonuçları ayrı ayrı değerlendirilmiştir.

Bu yöntem sayesinde hem hiperparametre seçiminin model performansı üzerindeki etkisi hem de Batch Normalization kullanımının öğrenme sürecine katkısı açık bir şekilde gözlemlenmiştir. Elde edilen bulgular, uygun hiperparametre seçiminin ve doğru model iyileştirme tekniklerinin makine öğrenmesi uygulamalarında önemli bir rol oynadığını göstermektedir.



Şekilde modelin genel çalışma süreci gösterilmiştir. Random Search yöntemi ile en uygun hiperparametreler belirlendikten sonra model iki farklı şekilde eğitilmiştir. Batch Normalization kullanılan modelin daha yüksek doğruluk elde ettiği diyagram üzerinde gösterilmiştir. Bu durum, Batch Normalization tekniğinin model performansını artırdığını açıkça ortaya koymaktadır.

4. Uygulama Alanları

Makine öğrenmesi modellerinde hiperparametre optimizasyonu ve Batch Normalization gibi teknikler, günümüzde birçok farklı alanda aktif olarak kullanılmaktadır. Bu yöntemler, model performansını artırarak daha doğru ve güvenilir sonuçlar elde edilmesini sağlar.

Görüntü işleme alanında, özellikle nesne tanıma ve yüz tanıma sistemlerinde bu teknikler yaygın olarak kullanılmaktadır. Örneğin, güvenlik sistemlerinde kullanılan yüz tanıma algoritmalarında modelin hızlı ve doğru öğrenmesi büyük önem taşımaktadır. Bu noktada Batch Normalization, modelin daha stabil çalışmasını sağlarken, hiperparametre optimizasyonu en uygun model ayarlarının bulunmasına yardımcı olur.

Sağlık sektöründe, hastalık teşhisi ve tıbbi görüntü analizi gibi uygulamalarda makine öğrenmesi modelleri önemli rol oynamaktadır. Röntgen ve MR görüntülerinin analizinde kullanılan modellerde doğru sonuç elde etmek hayati önem taşır. Bu nedenle modelin performansını artıran tekniklerin kullanılması kritik bir gerekliliktir.

Finans alanında, dolandırıcılık tespiti ve risk analizi gibi konularda makine öğrenmesi algoritmalarından yararlanılmaktadır. Bu sistemlerde yüksek doğruluk oranı elde etmek, yanlış kararların önüne geçilmesi açısından oldukça önemlidir. Random Search gibi yöntemler, en uygun parametrelerin hızlı bir şekilde bulunmasını sağlayarak model performansını artırmaktadır.

Bunun yanı sıra, öneri sistemleri (örneğin e-ticaret sitelerinde ürün önerileri) ve doğal dil işleme uygulamalarında da bu teknikler aktif olarak kullanılmaktadır. Kullanıcı davranışlarını analiz eden sistemlerde doğru tahminler yapılabilmesi için modelin iyi eğitilmesi gerekmektedir.

Sonuç olarak, hiperparametre optimizasyonu ve Batch Normalization teknikleri yalnızca teorik değil, aynı zamanda gerçek dünya problemlerinin çözümünde de büyük öneme sahiptir. Bu yöntemler sayesinde daha hızlı, daha doğru ve daha güvenilir makine öğrenmesi sistemleri geliştirilebilmektedir.

Hiperparametre optimizasyonu ve Batch Normalization tekniklerinin gerçek dünya uygulamaları ve katkıları

Alan	Uygulama	Random Search Katkısı	Batch Normalization Katkısı
Sağlık	MR ve röntgen analizi	En uygun model parametrelerini belirler	Daha stabil ve doğru teşhis sağlar
Güvenlik	Yüz tanıma sistemleri	En iyi doğruluk için parametre ayarı yapar	Daha hızlı ve kararlı öğrenme sağlar
Finans	Dolandırıcılık tespiti	Farklı model ayarlarını test eder	Modelin aşırı öğrenmesini (overfitting) azaltır
E-ticaret	Öneri sistemleri	Kullanıcı davranışına uygun model ayarı yapar	Tahmin doğruluğunu artırır
Otomotiv	Otonom araç sistemleri	En uygun karar mekanizmasını optimize eder	Sensör verilerinde daha dengeli öğrenme sağlar
Sosyal Medya	İçerik öneri algoritmaları	Farklı parametrelerle performans artırır	Daha hızlı eğitim ve daha iyi sonuç sağlar
Doğal Dil İşleme	Chatbot ve çeviri sistemleri	Dil modelinin en iyi ayarlarını bulur	Modelin daha stabil çalışmasını sağlar

5. Yapılan Mini Proje (Kodlama) Açıklaması

Yapılan çalışmanın açıklaması

Bu çalışmada Random Search yöntemi kullanılarak KNN modeli üzerinde hiperparametre optimizasyonu gerçekleştirilmiştir. Model farklı parametre kombinasyonları ile eğitilmiş ve en iyi sonucu veren parametreler belirlenmiştir.

Kullanılan veri seti

Bu çalışmada, sınıflandırma problemleri için yaygın olarak kullanılan Iris veri seti kullanılmıştır. Veri seti üç farklı sınıfa ait örneklerden oluşmakta olup her bir örnek dört farklı özellik içermektedir.

Modelin performansını değerlendirmek amacıyla veri seti eğitim ve test olarak ikiye ayrılmıştır. Ayrıca modelin daha sağlıklı öğrenebilmesi için veriler üzerinde standardizasyon (ölçeklendirme) işlemi uygulanmıştır.

Kullanılan yöntem

Bu projede hiperparametre optimizasyonu için Random Search yöntemi kullanılmıştır. Bu yöntem, belirlenen hiperparametre aralıklarından rastgele seçimler yaparak farklı kombinasyonları denemeye dayanır.

Random Search yöntemi, Grid Search yöntemine göre daha hızlı olması ve geniş parametre aralıklarını daha verimli şekilde tarayabilmesi nedeniyle tercih edilmiştir. Bu çalışmada farklı hiperparametre kombinasyonları denenmiş ve en yüksek doğruluğu sağlayan parametreler belirlenmiştir.

Kullanılan kodlamalar

Projede Python programlama dili kullanılmıştır ve uygulama Jupyter Notebook ortamında gerçekleştirilmiştir. Kodlar hücreler halinde düzenlenmiş ve her adım açıklayıcı metinler ile desteklenmiştir.

Kodlama sürecinde aşağıdaki işlemler gerçekleştirilmiştir:

- Veri setinin yüklenmesi
- Verinin eğitim ve test olarak ayrılması
- Veri ön işleme (ölçeklendirme) işlemi
- KNN modelinin kurulması
- Random Search yöntemi ile hiperparametre optimizasyonu
- Modelin eğitilmesi ve test edilmesi
- Sonuçların grafik ve tablo olarak gösterilmesi

Elde edilen sonuçlar ve program çıktıları

Random Search yöntemi sonucunda model için en uygun hiperparametreler belirlenmiştir. Yapılan denemeler sonucunda en iyi parametre kombinasyonu aşağıdaki gibi elde edilmiştir:

```
random_search = RandomizedSearchCV(
    estimator=model,
    param_distributions=param_dist,
    n_iter=10,
    cv=5,
    scoring="accuracy",
    random_state=42,
    n_jobs=-1
)

random_search.fit(X_train_scaled, y_train)

print("En iyi parametreler:", random_search.best_params_)
print("En iyi skor:", random_search.best_score_)
```

```
En iyi parametreler: {'weights': 'distance', 'n_neighbors': 11, 'metric': 'minkowski'}
En iyi skor: 0.9666666666666668
```

weights: distance
n_neighbors: 11
metric: minkowski

Bu parametreler ile elde edilen modelin çapraz doğrulama skoru yaklaşık olarak **0.96** olarak bulunmuştur.

Test Sonucu:

```
best_model = random_search.best_estimator_
y_pred = best_model.predict(X_test_scaled)

accuracy = accuracy_score(y_test, y_pred)
print("Test doğruluk oranı:", accuracy)
```

```
Test doğruluk oranı: 1.0
```

En iyi parametreler ile eğitilen modelin test verisi üzerindeki doğruluk oranı %100 olarak elde edilmiştir.

Performans Çıktısı:

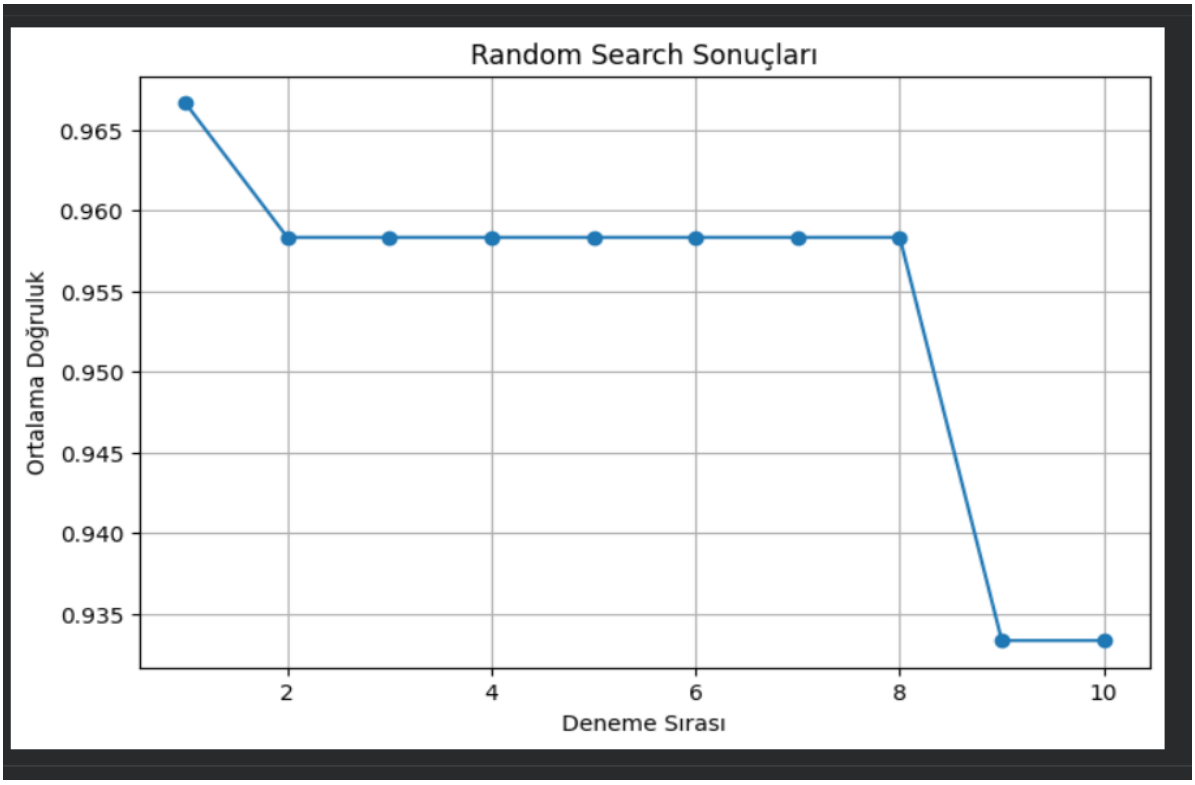
```python
print("Sınıflandırma Raporu:")
print(classification_report(y_test, y_pred, target_names=iris.target_names))

print("Karışıklık Matrisi:")
print(confusion_matrix(y_test, y_pred))
```

```
Sınıflandırma Raporu:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      1.00      1.00        10
   virginica       1.00      1.00      1.00        10

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30

Karışıklık Matrisi:
[[10  0  0]
 [ 0 10  0]
 [ 0  0 10]]
```

Modelin sınıflandırma raporu ve karışıklık matrisi incelendiğinde tüm sınıfların doğru şekilde tahmin edildiği görülmektedir.

Random Search Grafiği:

Farklı hiperparametre kombinasyonlarının model doğruluğu üzerindeki etkisi grafik ile gösterilmiştir.



6. Sonuç

Bu çalışmada Random Search yöntemi kullanılarak hiperparametre optimizasyonu gerçekleştirilmiştir. Elde edilen sonuçlar, doğru hiperparametre seçiminin model performansı üzerinde önemli bir etkisi olduğunu göstermektedir.

Random Search yöntemi sayesinde farklı parametre kombinasyonları hızlı bir şekilde denenmiş ve en uygun model parametreleri belirlenmiştir. Bu yöntem, özellikle geniş parametre aralıklarında etkili bir çözüm sunmaktadır.

Genel olarak değerlendirildiğinde, Random Search yönteminin makine öğrenmesi modellerinin performansını artırmada etkili bir yaklaşım olduğu sonucuna ulaşılmıştır.

Öğrenilenler:

Hiperparametre optimizasyonunun model performansı üzerindeki etkisi öğrenilmiştir.
Random Search yönteminin çalışma mantığı anlaşılmıştır.
Veri ön işleme (standardizasyon) işleminin model başarısına katkısı görülmüştür.
KNN algoritmasının nasıl çalıştığı ve nasıl optimize edildiği öğrenilmiştir.
Model performansını değerlendirmek için accuracy, classification report ve confusion matrix gibi metriklerin kullanımı öğrenilmiştir.
Jupyter Notebook ortamında kod yazma, çalıştırma ve sonuçları analiz etme becerisi geliştirilmiştir.

KAYNAKÇA

[1] I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.

[2] Scikit-learn, “Model selection and evaluation,” [Online]. Available: https://scikit-learn.org/stable/model_selection.html

[3] Scikit-learn, “RandomizedSearchCV Documentation,” [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.RandomizedSearchCV.html

[4] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.